

Performance modeling on DaVinci AI core

Yifeng Tang*, Cho-li Wang

Department of Computer Science, The University of Hong Kong, Room 414, Chow Yei Ching Building, Hong Kong

ARTICLE INFO

Article history:

Received 30 August 2022

Received in revised form 13 December 2022

Accepted 24 January 2023

Available online 1 February 2023

Keywords:

AI processors

Benchmarking

Performance modeling

ABSTRACT

The extensive use of Deep Neural Networks (DNNs) encourages people to design domain-specific hardware called Artificial Intelligence (AI) processors. The novel hardware makes optimizations challenging without a proper performance model that reveals working details and performance implications. This paper presents a performance model, Verrocchio, for Huawei DaVinci AI Core, which predicts the execution time of real-world DaVinci kernels. We propose specially-crafted micro-benchmarks to identify contention source, runtime behaviors, and bandwidth sharing, which significantly determine performance. Since DaVinci Core adopts a binary semaphore mechanism for synchronization, Verrocchio views each instruction as a discrete event and manages its execution time based on the programming logic. For evaluation, Verrocchio achieves average error rates of 2.62% and 2.30% in sample kernels for single-core and double-core execution. We demonstrate an optimizing process of matrix multiplications with Verrocchio, achieving speedups of $1.70\times$ for operators and $1.53\times$ for applications and error rates of 5.06% and 5.25%.

© 2023 Elsevier Inc. All rights reserved.

1. Introduction

Deep Neural Networks (DNNs) have played an active and indispensable role in an increasing number of rising areas, including medical biology [37], renewable energy power [10], and autonomous car industry [30]. Since users always pursue better performance and fewer costs, hardware producers create domain-specific hardware called Artificial Intelligence (AI) processors or Machine Learning (ML) processors [13]. AI processors are equipped with the specialized AI acceleration unit, mostly the matrix multiplier-accumulators (MACs).

DaVinci [23] is a novel AI processor architecture provided by Huawei. Based on DaVinci architecture, Huawei released Ascend 310, a mini AI processor. Ascend 310 has two DaVinci Cores and offers 8 TFLOPS (FP16) or 16 TOPS (INT8) computation power with a low power cost of 8W. Recently, Huawei introduced a full-stack framework called CANN [16]. It allows programmers to design and implement custom DNN operators on the Ascend 310, enabling researchers to propose specific algorithms and optimize their applications [27,18].

DaVinci processors adopt the SIMD (Single Instruction Multiple Data) execution model, a combination of separate memory buffers and shared Unified Buffer, and a 16×16 tree-like matrix MACs, Cube Unit, as the AI acceleration unit. The unique architectural

structure makes former performance modeling work [19,26,32] on other AI processors ineffective and brings several challenges to modeling work. The first challenge is the hardware performance benchmarking of the separate memory buffers and related Memory Transfer Engine (MTE) Units. The DaVinci AI Core has complex on-core data paths that independently connect different memory buffers controlled by MTE Units. When multiple MTE Units execute concurrently, they share the data paths and compete for the limited bandwidth, which forms a contention and significantly lower performance. Compared with the previous studies on hardware resource contention analyses [15,29], which focus on the contention ratios when contention is fully in progress, the performance model further requires an analysis of the contention source and the entire runtime behaviors of the contention from the beginning to the end. The second is the analysis of the synchronization supported by a binary semaphore mechanism. Manually written instructions of the binary semaphore are inserted to maintain the instruction-level data dependencies among several concurrent hardware units. Since the statistical performance model or the bound and bottleneck analysis (e.g., roofline model [31]) does not consider those dependencies, which determines the whole kernel program execution time (e.g., whether two instructions overlap or not), the performance predictions or the optimization suggestions can be inaccurate. The relationships among instructions and the runtime behavior of each instruction should be carefully analyzed in the performance model.

This work proposes a performance model, Verrocchio, for Huawei DaVinci architecture AI processors. In addition to the reg-

* Corresponding author.

E-mail addresses: u3006567@connect.hku.hk (Y. Tang), clwang@cs.hku.hk (C.-l. Wang).

ular benchmarks for individual hardware unit performance, we develop micro-benchmarks to reveal the bandwidth contention details of the data paths, which are controlled by the MTE Units. We design specifically-crafted micro-benchmarks, which initially stall one of the hardware units for a period and then release it to intentionally control the entire runtime of the contention. By changing the benchmark parameters, we dissect the contention source and entire runtime behaviors for a deeper understanding and accurate modeling of the DaVinci hardware architecture. In addition, we implement benchmarks on the bandwidth sharing with different contention ratios and data paths.

Given a kernel program of DaVinci Cube-based Compute Engine (CCE) instructions, Verrocchio predicts the execution time of each instruction and the whole kernel program based on the discrete-event simulation. Verrocchio classifies the events (CCE instructions) into three types, normal events, Set Flag events, and Wait Flag events. Separately for the three types of events, we compute the event death time, which is the wall clock time when the event finishes. The Set Flag and Wait Flag events are the basic manually-inserted operations of the binary semaphore mechanism, which determine the programming logic of the DaVinci kernels and help Verrocchio manage all the events located at the concurrent hardware units accordingly.

This paper has three main contributions as follows:

- We demystify Huawei DaVinci architecture Ascend 310 and propose a discrete-event-based performance model Verrocchio. Verrocchio models a series of architectural features that are most important and determinant for the DaVinci Core performance, including bandwidth contentions and concurrent hardware unit execution with synchronization.
- We create micro-benchmarks to show the characteristics of the DaVinci Core hardware units. We reveal how MTE Units control and compete for the data paths which connect the different separate memories. In addition to the contention ratio benchmark on bandwidth sharing, we design specially-crafted benchmarks to identify the source, which is asserted to be the Interconnect Bus, and the runtime behaviors of the bus contention.
- We evaluate the accuracy of Verrocchio with various kernels. Verrocchio achieves an average error rate of 2.62% and 2.30% for the single-core and double-core execution of several sample kernels. Then we demonstrate the use of Verrocchio in optimizing a matrix multiplication algorithm. Verrocchio enables a search space exploration in tiling size selection and achieves a speedup of $1.70\times$ at the operator level and $1.53\times$ at the application level compared with CANN native operators with prediction error rates of 5.06% and 5.25% for the single-core and double-core execution.

2. Background, DaVinci architecture

With the recent rapid development of DNN applications, standalone AI processors with specialized AI acceleration units receive enormous attention. As shown in Table 1, we list the single-core details of several popular AI processors. While GPU manufacturers like Nvidia or Qualcomm utilize Single Instruction Multiple Threads (SIMT) to extend their mature GPU architectures, other AI processor manufacturers design each AI Core based on SIMD. Compared with the SIMT execution model, the SIMD model saves the power consumption, reduces the hardware scale, but loses programming flexibility. The memory hierarchy is generally divided into shared and separate memories. The shared memories aim to decrease the data transfer costs among hardware units, and the separate memories feature different bandwidths and sizes for specific usages. The designs of the AI acceleration units also vary a

lot. AI processors like Nvidia GPUs need a warp of 16 threads to execute one Tensor Core. Meanwhile, AI processors like Baidu Kunlun claim to equip a single-thread-driven MAC array. Google TPUs are built on a 256×256 systolic array, which requires a pipeline heating time and causes significant latency.

The Huawei DaVinci architecture draws on the strengths of others with its improvements based on the SIMD execution with a powerful Cube Unit. The DaVinci Core has several separate memory buffers supporting the Cube Unit and a shared Unified Buffer for other units. Multiple hardware components constitute DaVinci Core, including compute units, storage units, and control units, as shown in Fig. 1 (a). Each Ascend 310 processor has two DaVinci Core, sharing a 512-bits Interconnect Bus.

The compute units include the Scalar Unit, Vector Unit, and Cube Unit. The Scalar Unit does mostly address calculations or condition branch analyses. The Vector Unit supports vectorized arithmetic or logical computations, including data preprocessing and activations. The Cube Unit is the essence of the DaVinci Core, improving the computation power of the matrix multiplications. A Cube Unit consists of 4096 FP16 MACs and 8192 INT8 MACs, forming 16×16 tree-like multiplier-accumulators, as shown in Fig. 1 (c) (d). Each matrix multiplication is broken into multiple basic units of $16 \times 16 \times 16$ matrix multiplications. The accumulator behind the Cube Unit accumulates the intermediate results in the LOC Buffer.

The storage units refer to external storage, internal storage, and memory transfer units. The external storage includes L2 Buffer, DDR, and other storage devices accessed by the bus. The internal storage includes five memory buffers (L1, LOA, LOB, LOC, Unified Buffer) and registers. The Memory Transfer Engines (MTEs) control the data paths, which connect the internal storage buffers and transfer the data. The three MTEs (MTE1, MTE2, and MTE3) are tasked with the distinct data paths independently, as shown in Fig. 1 (b). The five buffers and registers have different capacities and bandwidths, filling the gap between the low bandwidth and the high compute speed.

The control units contain System Control, Scalar Program Scheduling Queue (PSQ), Instruction Dispatch, instruction queues (Cube Queue, Vector Queue, and three MTE Queues), and Event Synchronization. The Scalar PSQ receives and decodes the executing instructions from the Instruction Cache. After decoding, the Instruction Dispatch transmits the instructions to the queues separately. The Event Synchronization handles the data dependency among the units to ensure the correctness of execution orders, mainly a binary semaphore mechanism with eight registers.

3. Performance modeling on DaVinci cores

The specific hardware structure of the DaVinci Core guarantees high performance for AI applications while leaving several challenges for performance modeling and further optimization. This section first discusses the key challenges we address, how to model the storage unit hardware performance and the concurrent execution synchronization. Then we give an overview of our performance model, Verrocchio, which aims to overcome the challenges for accurate performance modeling.

3.1. Key challenges

3.1.1. Storage unit hardware performance

Since different MTE Units control different data paths, the bandwidth of each path requires benchmarking respectively. Especially, multiple MTE Units are allowed to share a single data path concurrently, which would bring bandwidth contentions and significantly influence the hardware performance. As shown in Fig. 1 (a), the data path controlled by the on-core bus is the only access for the DaVinci Cores to the off-core 512-bits Interconnect Bus. Fig. 1 (b)

Table 1
Specifications of current popular AI processors for single-core details.

AI Processor	Exec. Model	Memory Hierarchy	AI Acceleration Unit
Qualcomm Hexagon 780 [25]	SIMT	Seamless shared memory for all acceleration units	Fused AI Accelerator including Scalar, Tensor & Vector
Google TPU V4 [20]	SIMD	Unified buffer as local storage DRAM Chips as weights inputs	Matrix Multiply Unit based on 256×256 systolic array
Cambricon MLU290-M5 [2]	SIMD	Scratchpad memories for units separately w/DMA to IO	Neural Functional Units succeeding to DianNao [3]
Nvidia A100 GPU [5]	SIMT	GPU memory hierarchy w/Tensor Core register files	16 Dot Product units operated by one warp w/an MMA instruction [19]
Baidu Kunlun K200 [24]	SIMD	Separate buffers for inputs, and outputs after activation units	A MAC array w/ activation & element-wise units
Huawei DaVinci Ascend [23]	SIMD	Separate L1, L0 buffers for Cube Unit, Unified Buffer for others	Cube Unit based on 16×16 tree-like MACs

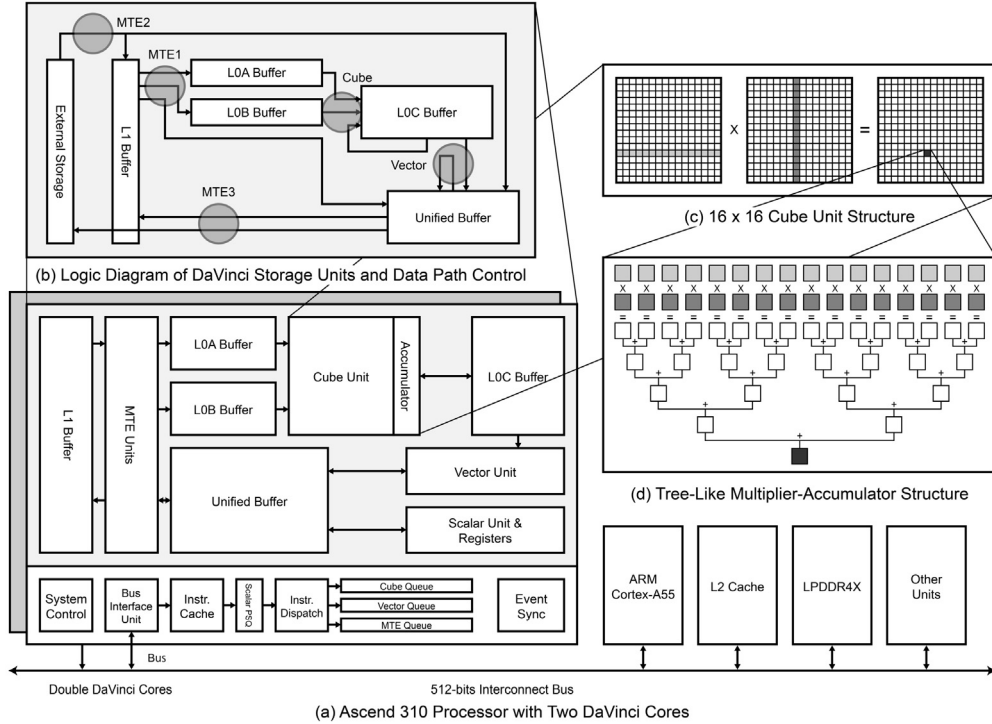


Fig. 1. Overall Ascend 310 Structure. (a) Ascend 310 Processor with Two DaVinci Cores, (b) Logic Diagram of DaVinci Storage Units and Data Path Control, (c) 16×16 Cube Unit structure, (d) Tree-Like Multiplier-Accumulator Structure (from [23]).

illustrates that the MTE2 and MTE3 Units both transfer data between the external storage and the DaVinci Core. The architecture suggests a potential contention between the MTE2 and MTE3 Units. However, from the figure of the hardware structure, we cannot postulate the existence of the contention and figure out the contention source: the MTE Units, the on-core bus unit, or the off-core Interconnect Bus. Without the knowledge of the source, simply fitting the performance of the contentions could be inappropriate.

Furthermore, existing studies on the hardware resource contentions [15,29] mainly focus on the performance variations caused by the different contention ratios when the bandwidths are saturated. In addition to the analyses of the contention ratio, the benchmark of the MTE Unit contention in our performance modeling should pay further attention to the entire runtime of the contention and not only to the stable saturated status. One of the most common methods to monitor the entire runtime of a kernel is sampling [17,7,11]. However, since the DaVinci Cores have no breakpoint or in-kernel timestamp mechanism, we can hardly insert sampling points into the DaVinci kernels. Therefore, designing a benchmark to measure the runtime of the contention becomes a challenge.

3.1.2. Concurrent execution and synchronization

As we demystify in Fig. 2, all the instruction queues, including the Scalar PSQ, strictly follow the FIFO policy. The Scalar PSQ maintains input kernel instructions and identifies the type of each instruction (Scalar, Vector, Cube, MTE1/2/3), which performs as Instruction Decoder in CPUs. The types of instruction determine the instruction queue to which the instruction is sent, as well as the concurrent unit on which the instruction is executed. For the Scalar instructions, the Scalar PSQ directly sends them to the Scalar Unit. For others, the Scalar PSQ sends them to the corresponding instruction queues via Instruction Dispatch. Each compute or storage unit executes one single instruction at every moment. When a unit is idle, the corresponding queue dequeues the oldest instruction to the unit for execution. The Scalar PSQ asynchronously dispatches the instructions except for the Scalar instructions.

While the DaVinci Core adopts the single-thread execution model, the concurrency of the hardware units makes the Instruction-Level Parallelism (ILP) supported. To guarantee the dependency among the units, the programmers must manually insert the `set_flag` and `wait_flag` instructions, acting as the binary semaphore mechanism. The fundamental operations of the

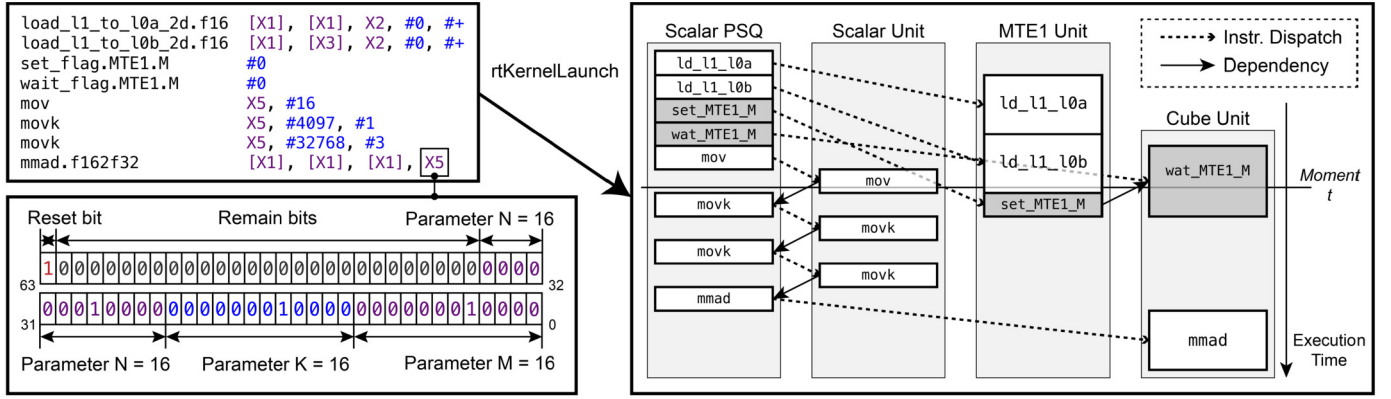


Fig. 2. The DaVinci Core working details.

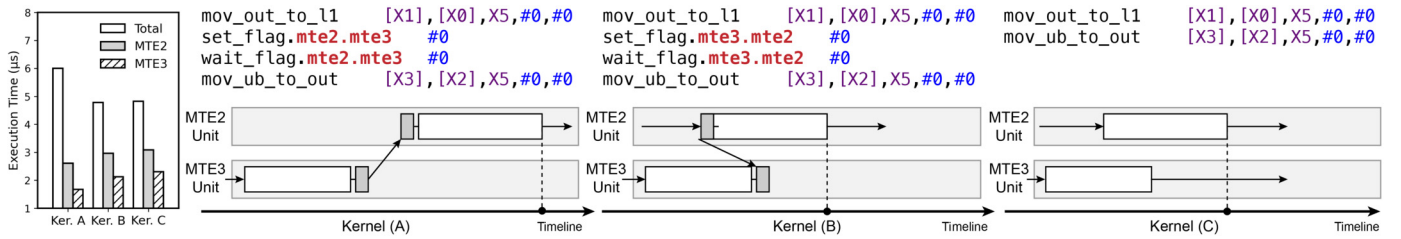


Fig. 3. The examples of DaVinci kernels influenced by the binary semaphore.

semaphore mechanism are PV Operations [9]. P Operation decrements the value of the semaphore variable by 1. If the new variable is negative, it blocks the caller process. Meanwhile, V Operation increments the value of the semaphore variable by 1. If the old variable is negative, it wakes up a blocked process and lets it access a resource unit. In DaVinci Cores, the `wait_flag` acts as P Operation and the `set_flag` performs as V Operation. The Scalar PSQ assigns the two semaphore operations to the corresponding instruction queue just as it does with other instructions. In the example of the moment t , following the FIFO policy, the `set_flag` (`set_MTE1_M`) in the MTE1 queue shall not start due to the incompleteness of the last instruction `ld_l1_l0b`. The corresponding `wait_flag` (`wat_MTE1_M`) keeps blocking the Cube Unit until the execution of `set_flag`. Therefore, the instruction `mmad` shall not start before the end of the `ld_l1_l0b`.

The idleness of the hardware unit caused by the synchronization is one of the most critical factors influencing the execution time. Each instruction could prolong or block the execution based on its programming logic. Furthermore, the concurrent execution can be intentionally activated or deactivated, which is decided by how the kernel programs schedule the multiple semaphore registers and control flows. Fig. 3 illustrates a simple DaVinci kernel containing two data transfer operations (on MTE2 and MTE3 Units) with different binary semaphore operations. As a result, Kernel (A) takes $1.26\times$ more time than Kernel (B) and $1.24\times$ more than Kernel (C), while the actual two data transfer operations take similar time among all kernels. For Kernel (A) and Kernel (B), the parameter order (`mte2.mte3` & `mte3.mte2`) of the semaphore operations determines the execution time. The parameter order `mte2.mte3` prohibits the concurrent execution while the reversed `mte3.mte2` does not, which reports a similar result to Kernel (C). Therefore, the DaVinci Core working details make the modeling methods based on kernel metadata analysis or statistics ineffective, e.g., the most straightforward execution time computed by *DataSize/Bandwidth*. From the view of those modeling methods

without analysis in the instruction logic, Kernel (A) and (B) should have a similar total execution time, which is false in our evaluations. Then it is necessary to model the performance with the instruction-level analyses in the kernel source codes.

3.2. Overview of Verrocchio

Verrocchio is a discrete-event-based performance model, where events occur due to occurrence time with execution periods at the discrete points in time. Fig. 4 illustrates the structure of our DaVinci Core model. Verrocchio has two inputs, the target program source codes and the hardware benchmark results. It predicts and outputs the execution period of the kernel program.

To provide instruction-level modeling for the kernel program, Verrocchio first scans the instructions of the kernel program to abstract them into discrete events. It also collects the data size for each instruction. Verrocchio applies the results from our instruction-level micro-benchmarks, which target to expose the hardware performance, especially the DaVinci Core storage unit performance. Combining the data size and the benchmark results, Verrocchio estimates the static execution period of each instruction, which is the execution period when the instruction executes alone.

Two core components of Verrocchio are the dependency analyzer and event dispatcher, which manage the prepared discrete events and place them on a DaVinci execution timeline at the correct timestamp with a wall clock. To handle the synchronization supported by the binary semaphore, Verrocchio maintains two arrays from the analysis of the prepared events. The two sorted arrays store the Set Flag and Wait Flag operations and guide Verrocchio correctly start an event when meeting the binary semaphore conditions. As we discussed, during the procedure, the event execution periods can be updated because of the bandwidth contention, as the example of Event 5 shown in Fig. 4. To model the contentions, Verrocchio offers the execution periods adjust-

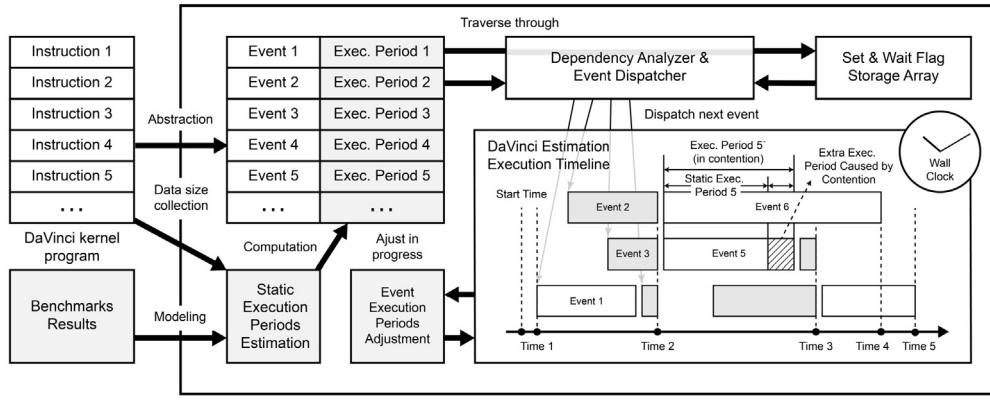


Fig. 4. Verrocchio, a discrete-event-based performance model for DaVinci Cores.

Table 2

Ascend 310 Cube Unit benchmark results.

Benchmark Types	Double-Core	Single-Core	Percentages
MTE1 bw. (L1 Buffer to LOA Buffer)	695.97 GB/s	347.99 GB/s	99.99%
MTE1 bw. (L1 Buffer to LOB Buffer)	348.79 GB/s	174.37 GB/s	100.00%
Vector bw. (None)	348.16 GB/s	174.06 GB/s	100.00%
Vector bw. (F32 to F16)	348.2 GB/s	174.09 GB/s	100.00%
Cube Unit practical FLOPS (RESET off)	10796.92 GFLOPS	5390.32 GFLOPS	100.00%
Cube Unit practical FLOPS (RESET on)	10789.17 GFLOPS	5397.34 GFLOPS	99.99%
Ascend 310 documented FLOPS	-	4096.00 GFLOPS	-
Kernel launch time	2293.5 ns	2354.5 ns	-

ment, which updates the new execution periods each time the contentions begin and end.

4. Benchmarking DaVinci

To reveal the performance of the storage unit, we benchmark the Ascend 310 processor by launching a series of instruction-level micro-benchmarks. For the hardware units, whose performance reports a regular and stable trend, the benchmark increases the processed data size and computes the bandwidth by the least-square method. For the bus bandwidth, which connects to the external storage and involves the contention by the MTE2 and MTE3 Units, we first design a specifically-crafted benchmark to identify the contention source and expose the runtime behaviors. Then we adopt further benchmarks with different contention ratios to quantify the bandwidth sharing under contentions.

4.1. Benchmarks for regular hardware units

4.1.1. Methodology

For the MTE1 Unit, we directly adopt the micro-benchmark kernels. For the Vector Unit, we list the data movement bandwidth here and with a precision converting function from FP32 to FP16. For the Cube Unit, we benchmark the FLOPS by modifying the `mmad` instruction parameters. The computation amount of each minimized $16 \times 16 \times 16$ matrix multiplication is $(15 + 16) \times (16 \times 16) = 7936$ FLOPs. After the single-core benchmarks, we activate and benchmark two DaVinci Cores on the Ascend 310 processor. We compare the results with the single-core results to check whether the double-core execution can approximately double the bandwidths or FLOPSs. In addition, we also benchmark the kernel launch time of double-core execution compared with the single-core execution.

4.1.2. Benchmark results

We list the benchmark results in Table 2. We summarize two main observations of the results as follows:

- As we expected, the independent hardware units report perfectly double acceleration. These hardware units require on-core resources only (bandwidths and computation power) and shall not suffer performance loss in the multi-core execution situation because of the Interconnect Bus.
- The kernel launch time results report that the double-core execution shows no apparent kernel launch time extension compared with the single-core execution. We assert that the kernel launch time does not suffer from any contentions in double-core execution. As DaVinci Cores have no inter-core communication mechanism, we assume that the two DaVinci Cores on Ascend 310 processors start and execute the kernel synchronically.

4.2. Benchmarks for bus contention source and runtime behaviors

While the bandwidth of the individual bus execution can be evaluated naturally with the benchmark in Sec. 4.1, the contention caused by the multiple units is non-trivial to be measured. In this subsection, we design a benchmark to study the source and the runtime behaviors of the contentions observed at the DaVinci Cores bus, which are caused by the MTE2 and MTE3 Units.

4.2.1. Benchmark design

The objective of the benchmark is to monitor the behaviors of the entire contention runtime, from the start to the end of it, with a fixed contention ratio of two (the MTE2 and MTE3 Units). As shown in Fig. 5, on one of the hardware units (e.g., the MTE2 Unit), we first let the unit process an increasing size of data, S_e , and then keep it idle until the end. For the other hardware unit (e.g., the MTE3 Unit), which is concurrently executing alongside, it first stays idle for a short period T_s and then processes a segment of data with the size of S_f until the end. On the DaVinci Cores, we insert dummy Scalar instructions between the instructions of the former unit and the latter, as annotated in Algorithm 1. As discussed, the Scalar PSQ must wait until the end of the Scalar instructions and

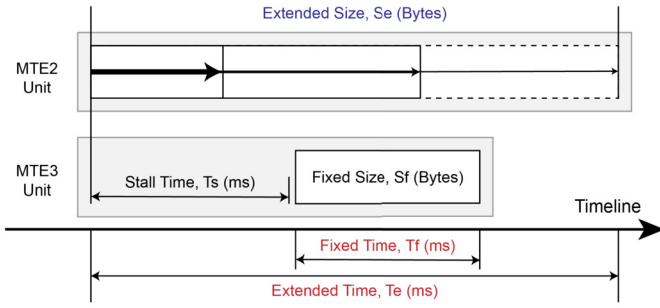


Fig. 5. Bandwidth contention monitoring benchmark (e.g., MTE2 & MTE3 Units).

Algorithm 1: Bandwidth Contention (e.g., MTE2 & MTE3 Units).

input : Stall time, T_s ; Fixed size, S_f
output: Extended time array: A_e ; Fixed time array: A_f

```

1 for  $S_e : 0 \rightarrow MAX$  do
2   Barrier call
3   (MTE2) Process data of size  $S_e$ 
4   (Scalar) Dummy instruction
5   (Scalar) ... (Block instruction 7 for  $T_s$ )
6   (Scalar) Dummy instruction
7   (MTE3) Process data of size  $S_f$ 

```

proceed to the next instruction. Therefore, the dummy instructions block the Scalar PSQ and keep the latter unit idle. When the former unit has been executing for a period, the latter unit finally receives its instruction from the Scalar PSQ and then executes with the designed contention. We measure and record the execution time of two units, T_e and T_f with the Huawei official profiler respectively, with the increment of S_e .

Divided by T_e , the two hardware units experience three contention statuses: before, under, and after contention. When $T_e < T_s$, where S_e is not large enough, the two hardware units stay the status before contention. When $T_e < T_s + T_f$, the two units are under contention. When $T_e > T_s + T_f$, the two units finish the contention periods at the status after contention. Therefore, after finishing the benchmark in Algorithm 1, the recorded execution time T_e and T_f can expose the entire runtime behaviors during the bandwidth contention. In addition, we vary the input parameters T_s and S_f to study the potential influences by the contention beginning time and processed data size under contention.

4.2.2. Contention source and runtime behavior results

Based on the fundamental benchmark designed in Sec. 4.2.1, we illustrate the benchmark results of the MTE2 and MTE3 Units with the different corresponding data paths in Fig. 6. We insert NOP instructions as dummy instructions, the number of which (related to T_s) are annotated in the legend of figures.

We first call the instructions for transferring data between the external storage and the L1 Buffer (MTE2) and that between the Unified Buffer and L1 Buffer (MTE3) concurrently, which does not involve other data paths. The results in Fig. 6 (a) show that with the variation of S_f , S_e , and T_s , the evaluated execution time T_f and T_e remain unchanged, which indicates no contention happens between the two MTE Units.

By contrast, from the results in Fig. 6 (b) and (c), we observe apparent irregular execution time variations, which present clear bandwidth contention runtime behaviors. For both the MTE2 and MTE3 Units, it is easy to identify the turning points of the three contention statuses we described in Sec. 4.2.1. With the increment of the data size S_e , the MTE2 Units report two different slopes in three-segment polylines, where the first segment is before contention, the second is under contention, and the third is after con-

tention. For the MTE3 Units with a fixed data size S_f , which were supposed to take a stable execution time, the contentions bring an extra execution time increment and also result in three-segment polylines. The number of the NOP instructions determines T_s and the length of the first segment. The increment of the MTE3 fixed data size S_f extends the second segment, which shows a lower bandwidth because of the sharing of the limited bandwidth under contention. The third segment's slopes are the same as the first for both the MTE2 and MTE3 Units, indicating that the bandwidths recover from the contention without extra side effects.

For the contention source, since it is impracticable to transfer the data without the off-core Interconnect Bus independently, the results shown in Fig. 6 (b) are introduced by both the on-core buses and the off-core Interconnect Bus. Meanwhile, Fig. 6 (c) involves the off-core Interconnect Bus only by placing the two instructions on two DaVinci Cores separately. In this way, in Fig. 6 (c), the on-core bus of Core 0 transfers data to the DaVinci Core with the MTE2 Unit individually and avoid any possibilities for contentions. Comparing the results of Fig. 6 (b) and (c), the execution time, the variation trends, and the slopes are similar, which suggests they activate the same contention source. Therefore, we assert that the contention source of the Ascend 310 processor bus bandwidth is the common part of Fig. 6 (b) and (c), the off-core Interconnect Bus.

Fig. 7 shows the results of the second benchmark, where we repeat the benchmark shown in Fig. 6 (b) but increase T_s with tiny steps to explore the execution patterns of the MTE Units under contention. Fig. 7 (a) plots the fixed-sized MTE2 Unit execution results ($S_f = 32$ KB) with the increasing MTE3 Unit data size S_e while Fig. 7 (b) is symmetric. With the increment of T_s , the beginning of the contention delays, which increases the start points of the second segment we discuss in Sec. 4.2.1 accordingly. However, from the results for the two cases, we observe that although T_s grows evenly, the start points of the second segments are not averagely distributed along the x-axis but gather at specific points, especially for the MTE3 Unit. The execution patterns show that the MTE Unit does not start the data transfer and forms a contention instantly after execution, suggesting the data is transferred in blocks or segments. We assert that before block transferring, the Interconnect Bus must manage and decide how to share the bandwidth based on the number of executing Units each time.

4.3. Benchmarks for bandwidth sharing under contention

After identifying the contention source and runtime behaviors, the maximum contention ratio ($R_{max} = 4$) and the participated hardware units (the MTE2 & MTE3 Units on two DaVinci Cores) are also determined. We further analyze the detailed policy for the potential participants to share the bandwidth.

4.3.1. Methodology

We follow the benchmark used in Sec. 4.1, where we increase the processed data sizes and compute the bandwidths under contention using the least-square method. We control the active hardware units, which are involved in the contentions of each case, with different instructions on two DaVinci Cores, respectively. Especially for the MTE3 Units, the benchmarks transfer the data from the Unified Buffer to the external storage, similar to Fig. 6 (b).

4.3.2. Bandwidth sharing results

Fig. 8 illustrates the benchmark results for six cases, which include all potential cases under contention. The bandwidth result of each MTE Unit plotted in the chart is the bandwidth of a single operation. Therefore, for the example in Fig. 8 (e), the total bandwidth is computed by $2 \times bw_{MTE2} + bw_{MTE3}$. Generally, with the increment of the contention ratio, the total bandwidth (MTE2 +

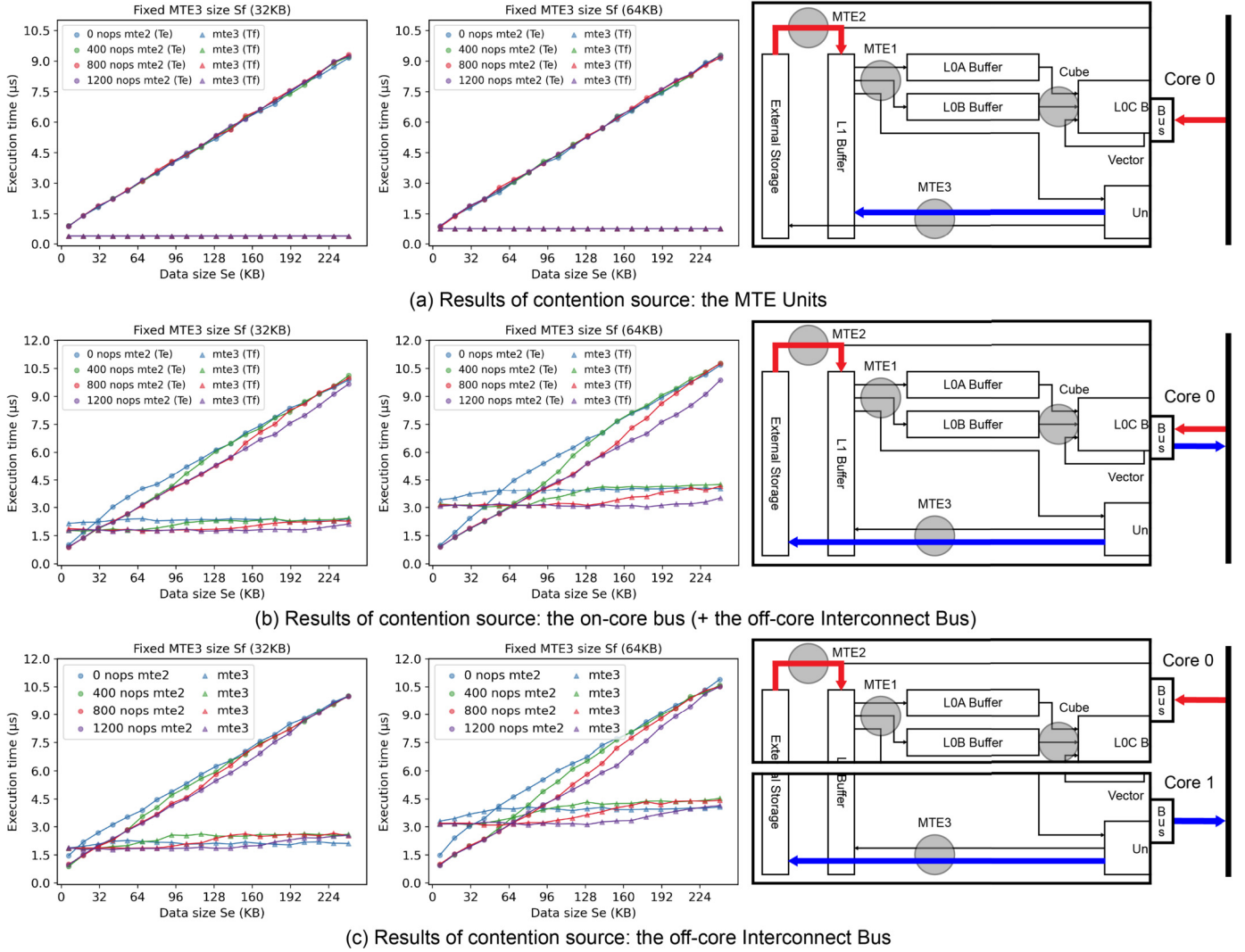


Fig. 6. Contention source results with the corresponding data paths.

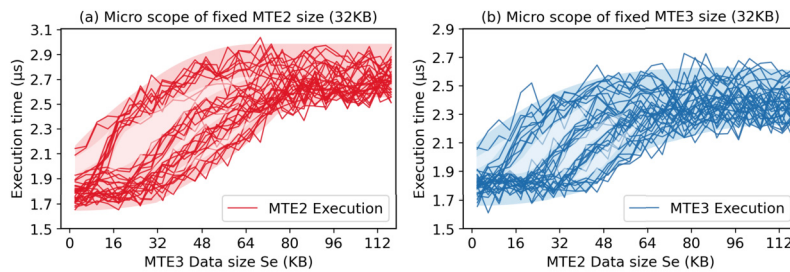


Fig. 7. The execution of the MTE Units with tiny T_s increments.

MTE3 Units) also grows with a maximum bandwidth of about 42 GB/s when the contention ratio is 4, shown in Fig. 8 (f).

The bandwidths of writing and reading operations, controlled by the MTE2 and MTE3 Units, are almost equal in all cases while sharing. When the contention ratio is 2, Fig. 8 (b, c, d) present three different combinations of the MTE operations, MTE2 and MTE3 operations on Core 0, MTE2 and MTE3 operations on Core 0 and 1, and two MTE2 operations on Core 0 and 1. In contrast to the different units participating, the bandwidth sharing results are the same. Either operation occupies half the total bandwidth in each case, no matter what kind of operation it is or which core the operation is executing. Fig. 8 (e) reports the bandwidth results

when the contention ratio is 3, where we place two MTE2 operations on two DaVinci Cores with an MTE3 operation on Core 0. The bandwidth of the MTE2 operation is still equal to the MTE3 operation, which implies that the three operations trisect the bandwidth of the Interconnect Bus. Fig. 8 (f) also shows a similar averagely sharing result. Therefore, we assert that the Interconnect Bus is a half-duplex bus, where the writing and reading operations can execute concurrently but influence each other. Our later bandwidth contention modeling work is based on the conclusion of the benchmark, where the bandwidth is shared equally by each Unit.

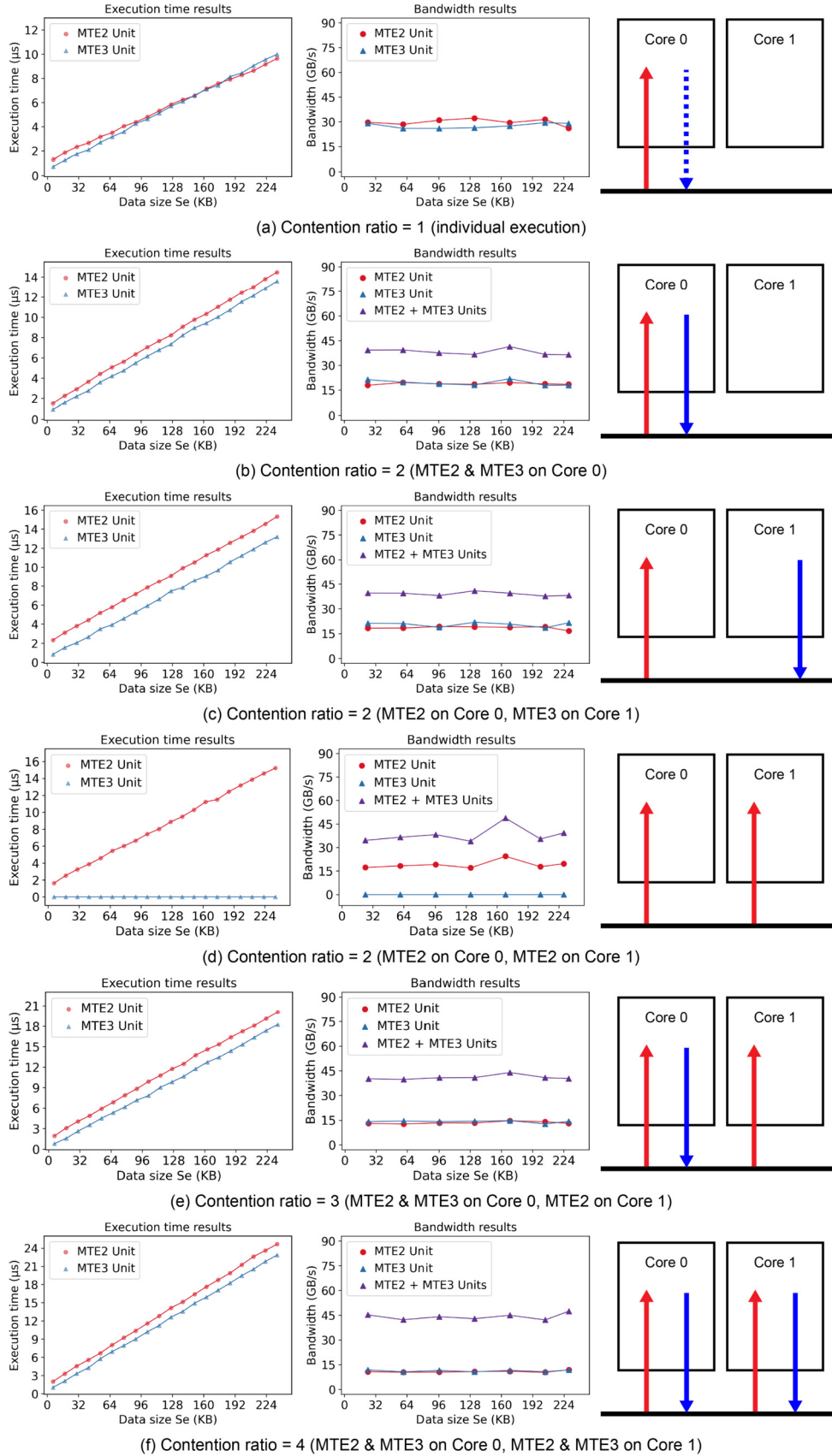


Fig. 8. Bandwidth sharing of the Ascend 310 processors under contention.

Table 3
Notations used in Verrocchio Performance Model.

Notations	Description	Value
u	The unit number	$0 < u \leq U$
N_u	The instruction number of the unit u for a kernel program	$0 \leq N_u$
n	The order number of an instruction on the unit u	$0 < n \leq N_u$
$E_{u,n}$	A discrete event, an abstraction of the DaVinci instruction, which is the n -th event executes on the Unit u	(u_a, u_b, r)
$T_{u,n}$	The death time of $E_{u,n}$, which is the wall clock time when $E_{u,n}$ finishes	$0 \leq T_{u,n}$
$P_{u,n}$	The execution period of $E_{u,n}$	$0 \leq P_{u,n}$
C	The input instructions, an ordered array of a series of $E_{u,n}$	$\langle E_{u0,n0}, E_{u1,n1}, \dots \rangle$
$DS_{u,n}$	The operated data size of $E_{u,n}$	$0 \leq DS_{u,n}$
$SE_{u,n}$	The sorted array to store the subscripts (u, n) of all Set Flag operations with the same value (u_a, u_b, r)	$\langle (u0, n0), (u1, n1), \dots \rangle$
$WE_{u,n}$	The sorted array to store the subscripts (u, n) of all Wait Flag operations with the same value (u_a, u_b, r)	$\langle (u0, n0), (u1, n1), \dots \rangle$

Table 4
Verrocchio hardware parameter definitions of the Ascend 310 processors.

Parameters	Description	Value
U	The total unit number	5
r_{max}	The maximum binary semaphore register number	8
$Init$	The initialization time of each instruction	40 ns
T_{kernel}	The initialization time of DaVinci kernel	2050 ns (inc. 1000 ns for MTE2, 350 ns for MTE3)
Th_u	The throughput (or bandwidth) of the Unit u	Partially Listed in Table 2

5. Modeling DaVinci

After benchmarking the hardware units, this section formally describes the performance model, Verrocchio, especially how it manages the concurrent execution and synchronization.

5.1. Verrocchio notation and parameter definitions

We list the notations used in Table 3 and part of the hardware parameters used in Table 4. Each instruction of the kernel program C is considered as a discrete event $E_{u,n}$ with an operated data size $DS_{u,n}$. Since the kernel program is determined, the order of the instructions on each unit is confirmed. Verrocchio uses the pair of u, n to locate instructions. The wall clock updates the death time $T_{u,n}$ of each event.

5.2. Verrocchio main performance modeling procedure

We show the Main Performance Modeling Procedure in Algorithm 2. Since the wall clock time when the kernel program finishes is the wall clock time when the last event finishes, which is the largest death time $T_{u,n}$. We assert that the execution time of the kernel program C is the maximum death time of the last Event E_{u,N_u} among all Units, as shown in Algorithm 2 line 12.

For all Events $E_{u,n}$, we divide them into three categories, normal computation or memory operations, Set Flag operations for `set_flag` instructions, and Wait Flag operations for `wait_flag` instructions. The last two categories are the fundamental concepts of the binary semaphore mechanism. Different categories have different procedures to update its death time $T_{u,n}$. We give $E_{u,n}$ a 3-tuple (u_a, u_b, r) as its value to classify the categories, where u_a, u_b are the related Units of a Set Flag or Wait Flag operation, r is the corresponding semaphore register, as shown in Fig. 9 (a).

5.2.1. Normal computation or memory operations

For the normal operations, we set $E_{u,n} = (0, 0, 0)$, which means $E_{u,n}$ is not related to the binary semaphore mechanism. For each normal computation or memory operation $E_{u,n}$, we first compute

Algorithm 2: Verrocchio Main Performance Modeling Procedure.

```

input : Kernel program  $C = \langle E_{u0,n0}, E_{u1,n1}, \dots \rangle$ 
        Operated data size  $DS = \langle DS_{u0,n0}, DS_{u1,n1}, \dots \rangle$ 
output: Kernel wall clock finish time:  $T_{end}$ 

1   $(T_{u0,n0}, E_{u,n}) \leftarrow (T_{kernel}, E_{u0,n0})$ 
2  while  $E_{u,n}$  is not the last event in  $C$  do
3    case  $E_{u,n} = (0, 0, 0)$  do // normal operations
4       $P_{u,n} \leftarrow DS_{u,n} / Th_u + Init$ 
5       $T_{u,n} \leftarrow T_{u,n-1} + P_{u,n}$ 
6    case  $E_{u,n} > (0, 0, 0)$  do // set flag operations
7       $T_{u,n} \leftarrow T_{u,n-1}$ 
8    case  $E_{u,n} < (0, 0, 0)$  do // wait flag operations
9       $E_{u',n'} \leftarrow E_{u,n}$ 
10      $T_{u,n} \leftarrow T_{u',n'} \leftarrow T_{S_{-E_{u,n}}[Index(u,n), WE_{u,n}]}$ 
11   Bandwidth_Contention_Check( $E_{u,n}$ )
12    $E_{u,n} \leftarrow$  the next event in  $C$ 
13  $T_{end} \leftarrow \max_{0 < u \leq U} T_{u,N_u}$ 

```

the corresponding static execution period $P_{u,n}$ to quantify how long the $E_{u,n}$ could occupy the Unit u . In general, $P_{u,n}$ is computed as shown in Algorithm 2 line 4. According to the FIFO policy, the incidence of $E_{u,n}$ should be immediately after the death of $E_{u,n-1}$. Hence, we compute the death $T_{u,n}$ of $E_{u,n}$ as the sum of the last death time $T_{u,n-1}$ on the same Unit u and the event execution period $P_{u,n}$, as shown in Algorithm 2 line 5.

5.2.2. Set flag operations

For the Set Flag operations, we set $E_{u,n} > (0, 0, 0)$. For the example in Fig. 9 (a), the instruction `set_MTE1_M` is converted to $E_{MTE1,3} = (MTE1, M, 1)$, which means the MTE1 Unit has finished its work, and the Cube Unit should terminate its corresponding Wait Flag operation and continue its next operation with Register 1. Since the Set Flag operations do not suspend or finish any workloads, the death time $T_{u,n}$ equals the last death time $T_{u,n-1}$.

Before the Main Performance Modeling Procedure, we create and maintain an array $SE_{u,n}$ to store the Set Flag operations with the subscript of the 3-tuple (u_a, u_b, r) . We store the 2-tuple (u, n)

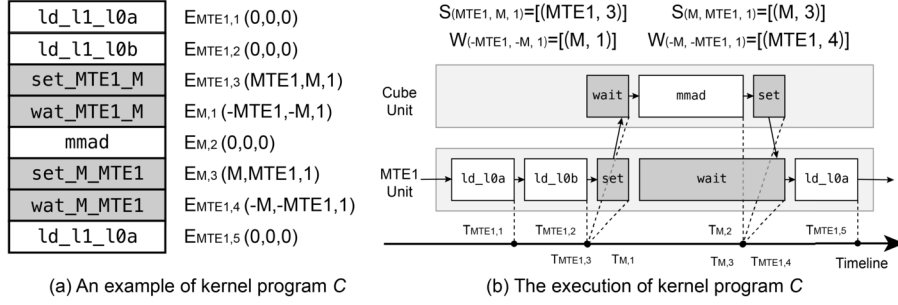


Fig. 9. An example of Verrocchio main performance modeling procedure.

of $E_{u,n}$, the identification of the Set Flag operation. We have the definition of $S_{E_{u,n}}$ shown below:

$$S_{E_{u_1,n_1}} = S_{E_{u_2,n_2}} = S_{(u_a, u_b, r)} = \langle (u_1, n_1), (u_2, n_2), \dots \rangle \quad (1)$$

$$\forall E_{u,n} > (0, 0, 0), n_i < n_j, \forall i < j$$

where the array $S_{E_{u,n}}$ is sorted according to the operation order number n .

5.2.3. Wait flag operations

For the Wait Flag operations, we set $E_{u,n} < (0, 0, 0)$, which is symmetric to the Set Flag operation definition. For the example in Fig. 9 (a), the instruction `wat_MTE1_M` is converted to $E_{M,1} = (-MTE1, -M, -1)$. We also create and maintain an array $W_{E_{u,n}}$ to store the 2-tuple (u, n) , as shown below:

$$W_{E_{u_1,n_1}} = W_{E_{u_2,n_2}} = W_{(-u_a, -u_b, -r)} = \langle (u_1, n_1), (u_2, n_2), \dots \rangle \quad (2)$$

$$\forall E_{u,n} < (0, 0, 0), n_i < n_j, \forall i < j$$

For a Wait Flag operation $E_{u,n}$, its corresponding Set Flag operation $E_{u',n'}$ must exist, where $E_{u,n} = -E_{u',n'}$, as shown in Algorithm 2 line 9. The array S and W are used to locate the related Set Flag and Wait Flag operation pair. Since DaVinci Cores have no memory heap or stack structure but only registers to store the semaphores, the Wait Flag operation order number in $W_{E_{u,n}}$ equals the order number of its corresponding Set Flag operation in $S_{E_{u',n'}}$. We introduce a function $Index(x, Array)$ to return the position of x in an array $Array$ without duplicated elements, then we have the relation:

$$Index((u, n), W_{E_{u,n}}) = Index((u', n'), S_{E_{u',n'}}) \quad (3)$$

$$= Index((u', n'), S_{-E_{u,n}})$$

Hence, we have:

$$(u', n') = S_{-E_{u,n}}[Index((u, n), W_{E_{u,n}})] \quad (4)$$

Since a Wait Flag operation blocks the Unit until the corresponding Set Flag operation is done, the death time of the Wait Flag operation equals the Set Flag operation death time. For the example in Fig. 9 (b), we compute $T_{M,1}$ as follows:

$$T_{M,1} = T_{S_{-E_{M,1}}[Index((M,1), W_{E_{M,1}})]} = T_{S_{(MTE1,M,1)}[0]} = T_{MTE1,3} \quad (5)$$

5.2.4. Bandwidth contention updating

As we discussed, the bandwidth contention of the Interconnect Bus influences the performance of the MTE2 and MTE3 Units significantly. The runtime behaviors we expose in Sec. 4.2.2 show that each time the two Units start to run concurrently, the contention occurs, and the related bandwidth changes; when either of the Units finishes, the contention ends, and the Units regain the original bandwidth. For the example in Fig. 10 (a), at $T_{MTE3,1}$,

when the MTE3 starts to execute the instruction `ld_ub_to_gm`, the contention begins; when the MTE2 Unit finishes its work at $T_{MTE2,1}$, the contention is finished. Therefore, the contention status changes at the event death time. Verrocchio checks whether a contention appears or disappears when an event is finished to update the death time of the related events.

We first compute the left data size of the event, which is influenced by the contention status change. For an event $E_{u,n}$, we compute the left data size as shown below:

$$Ds'_{u,n} = Ds_{u,n} \times \frac{T_{u,n} - T_{cur}}{P_{u,n}} \quad (6)$$

where T_{cur} is the current global time, also the death time of the latest event triggering the bandwidth contention, e.g., $T_{MTE3,1} = T_{cur}$ in Fig. 10 (a), $Ds'_{u,n}$ is the data size left for the new contention status. Then we update the event death time as follows:

$$T'_{u,n} = T_{cur} + Ds'_{u,n} / Th'_u \quad (7)$$

where Th'_u is the bandwidth of the Unit u at the new contention status, $T'_{u,n}$ is the new death time. For Th'_u , we strictly follow our conclusions in Sec. 4.3.2, where multiple Units equally share the bandwidth. The potential executions with a contention are divided into two cases shown in Fig. 10, based on the relationship of the two events' execution period. For the case in Fig. 10 (a), both units suffer one contention status change; for Fig. 10 (b), one unit suffers two contention status changes, experiencing the three-segment polyline of bandwidths discussed in Sec. 4.2.2.

5.3. Multi-core execution performance modeling

According to our benchmark results, the DaVinci hardware units can be classified into two categories, the one which relies on the independent on-core resources and the other which occupies the Interconnect Bus and causes contentions. For the first category, the throughput Th_{single} of every single core does not change (as a const value C_{single}) in the multi-core execution, and the total unit throughput Th_{total} of all cores increases proportionally.

$$Th_{single} = C_{single} \quad Th_{total} = N \times Th_{single} \quad (8)$$

where N is the number of the activated DaVinci Cores. For the second category, as our benchmarks report in Sec. 4.3.2, the throughputs of all single core Th_{single} averagely share the total throughput.

$$Th_{single} = \frac{Th_{total}}{N} \quad Th_{total} = C_{total} \quad (9)$$

Like most multi-core accelerators, we assume that multi-core DaVinci processors continue to adopt the SIMD execution model, which means the programs are tiled and distributed to multiple cores. We focus on a general normal computation or memory operation $E_{u,n}$, submitted to multiple DaVinci Cores, as shown in

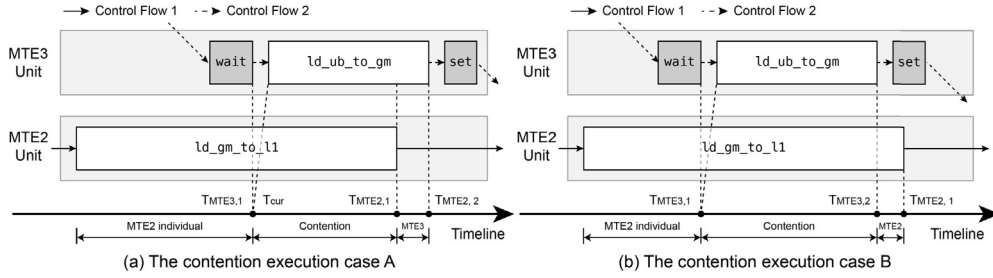


Fig. 10. Examples of Verrocchio contention execution.

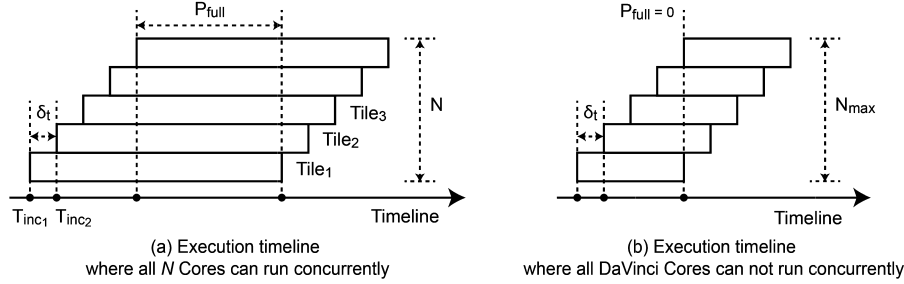
Fig. 11. Execution timeline of $E_{u,n}$, which is tiled and submitted to multiple DaVinci Cores.

Fig. 11, where each bar is a tile of $E_{u,n}$, e.g. $Tile_1$, $Tile_2$. To model the differences in the $E_{u,n}$ incident time $T_{u,n,inc}$ in all DaVinci Cores (e.g. $T_{inc2} - T_{inc1}$), we introduce δ_t , which can be measured in our benchmarks. It represents the average interval between the two nearest incident times among all DaVinci Cores.

For each tile of $E_{u,n}$ among all DaVinci Cores, the workloads (data sizes) DS_{single} are the same. Because of δ_t , the tile of $E_{u,n}$ on each core shows a stepped execution pattern with a start-up time. The execution period $P_{u,n}$ of $E_{u,n}$ for case (a) in Fig. 11 is calculated as below:

$$P_{u,n} = 2(N-1)\delta_t + P_{full} \quad (10)$$

where P_{full} is the period when all the cores are running concurrently. The execution period $P_{u,n}$ of $E_{u,n}$ for case (b) is calculated as below:

$$P_{u,n} = 2(N_{max}-1)\delta_t \quad (11)$$

where N_{max} is the maximized all-busy core number for the event $E_{u,n}$. Then, to calculate $P_{u,n}$, we must determine P_{full} and N_{max} first. For the first category of the units, which relies on the independent on-core resources only, we have Eq. (12) for the case (a) in Fig. 11:

$$2Th_{single}\delta_t \times \sum_{i=1}^{N-1} i + N \times Th_{single}P_{full} = N \times DS_{single} \quad (12)$$

$$\Rightarrow P_{full} = \frac{DS_{single} - (N-1)Th_{single}\delta_t}{Th_{single}}$$

Fig. 11 shows that, the condition converting the case (a) to the case (b) is $P_{full} = 0$, which means that all DaVinci Cores cannot run concurrently and then the core number N is N_{max} . Then we have the relation shown below:

$$P_{full} = \frac{DS_{single} - (N_{max}-1)Th_{single}\delta_t}{Th_{single}} = 0 \Rightarrow N_{max} = \frac{DS_{single}}{Th_{single}\delta_t} + 1 \quad (13)$$

We follow a similar idea for the second category of units as for the first category units. Hence, we list the equation below:

$$2Th_{total}\delta_t \times (N-1) + Th_{total}P_{full} = N \times DS_{single}$$

$$P_{full} = \frac{NDS_{single} - 2(N-1)Th_{total}\delta_t}{Th_{total}}$$

$$N_{max} = \frac{2Th_{total}\delta_t}{2Th_{total}\delta_t - DS_{single}} \quad (14)$$

The Ascend 310 processors are the simplest DaVinci architecture processors and have two DaVinci Cores, and then we have $N = 2$. According to our benchmark results, as shown in Table 2, $\delta_t \approx 61ns$, which is 2.5% of the kernel launch time and can be ignored. Hence, in the double-core execution, the total time can be simplified to $P_{u,n} = P_{full} = DS_{single}/Th_{single}$ for the first category of units or $P_{u,n} = N \times DS_{single}/Th_{total}$ for the second category.

6. Evaluation

This section evaluates the accuracy of Verrocchio. We first implement several basic kernels for the accuracy evaluations. Then as a complicated test case and a demonstration of the usage of Verrocchio, we show a matrix multiplication optimizing process guided by Verrocchio. All the experiment results are collected by Huawei's official profiler on the Ascend 310 processor (two DaVinci Cores) with Huawei Kunpeng 920 2.6 GHz CPU equipped.

6.1. Sample kernel evaluation

Table 5 lists the sample kernels we implement for evaluations. The cases include memory transfers, single computations, and several basic applications. We evaluate the accuracy results for both single-core and double-core situations. Fig. 12 illustrates the evaluated error rate results for each case.

Generally, Verrocchio reports high accuracy results. For the single-core execution, the average error rate is 2.62%; for the double-core execution, the result is 2.30%. The error rates among all cases do not show a system error but are randomly scattered around the baseline. The single-core kernel *mte1B* reports the maximum error rate of 7.33%. On the other hand, for the double-core

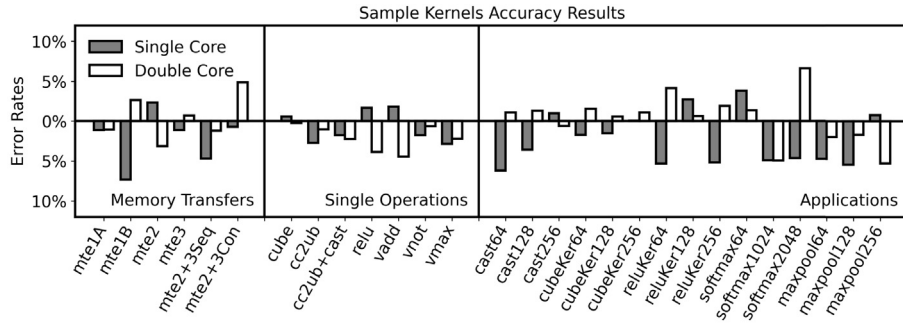


Fig. 12. The Verrocchio error rate results for sample kernels.

Table 5
Verrocchio sample kernels for accuracy evaluation.

Categories	Kernel Name	Description
Memory transfers	mte1A	Transfer data from L1 Buffer to LOA Buffer
	mte1B	Transfer data from L1 Buffer to LOB Buffer
	mte2	Transfer data from external storage to L1 Buffer
	mte3	Transfer data from Unified Buffer to external storage
	mte2+3Seq	MTE2 and MTE3 Units execute sequentially
	mte2+3Con	MTE2 and MTE3 Units execute concurrently
Single computations	cube	Do a matrix multiplication w/ Cube Unit
	cc2ub	Transfer from LOC Buffer to Unified Buffer w/ Vector Unit
	cc2ub+cast	Same as above plus type conversion
	relu	ReLU activation function
	vadd	Vectorized addition
	vnot	Vectorized boolean NOT operation
	vmax	Vectorized element-wise Max operation
Applications (Full kernels with I/O)	castXxx	Type conversion operator
	cubeKerXxx	Matrix multiplication operator without tiling
	reluKerXxx	ReLU activation operator
	softmaxXxx	Softmax operator
	maxpoolXxx	MaxPooling operator

execution, *mte1B* reports a much lower error rate of 3.14%. Since the MTE1 Unit is the independent on-core unit we discussed in Sec. 5.3, which does not suffer the performance loss in the multi-core execution, *mte1B* is expected to show similar error rates in the two situations. Therefore, the practical results suggest that the MTE1 Unit is not stable as other independent on-core units, which may require deeper analyses. In addition, we observe that the error rates of the Application kernels and the Memory Transfer kernels are higher than the Single Operation kernels, which indicates that the memory transfers, especially between the on-core buffers and the external storage, are the main prediction error sources of Verrocchio.

6.2. Matrix multiplication optimizing process

This section demonstrates an optimizing process of matrix multiplication with Verrocchio, one of the most significant applications for AI processors.

6.2.1. Improved matrix multiplication and tiling parameter selection

Algorithm 3 shows the target improved matrix multiplication algorithm for optimization. The matrix multiplications require four tiling parameters, *mTiles*, *kTiles*, *nTiles*, and *bNum*, which represents the tile number from the *m*, *k*, and *n* directions with the multiple control flow number.

The matrix multiplication adopts a policy called Double Buffer or Ping-Pong Buffer, shown in Algorithm 3 line 3. DaVinci Core buffers allow reading and writing on different addresses at the same buffer concurrently. Meanwhile, the binary semaphore mechanism allows at most eight independent control flows. Hence, the Double Buffer policy splits the buffers and the operations in half

Algorithm 3: Target Improved Matrix Multiplication.

```

input : matrix A, B of size  $(m \times k)$ ,  $(k \times n)$ 
        tiling parameter mTiles, kTiles, nTiles, bNum
output: matrix C of size  $(m \times n)$ 

1 (mSlice, kSlice, nSlice)  $\leftarrow (m, k, n) / (mTiles, kTiles, nTiles)$ 
2 for i  $\leftarrow 0$ , j  $\leftarrow 0$  to mTiles, nTiles do
3   for l  $\leftarrow 0$ , bf  $\leftarrow 0$  to kTiles / bNum, bNum do
4     if j = 0 then
5       | load A tile mSlice  $\times$  kSlice to L1 from External ; // MTE2
6       | load A tile mSlice  $\times$  kSlice to LOA from L1 ; // MTE1
7       if i = 0 then
8         | load B tile kSlice  $\times$  nSlice to L1 from External ; // MTE2
9         | load B tile kSlice  $\times$  nSlice to LOB from L1 ; // MTE1
10        | do matrix multiplication of A tile  $\times$  B tile to LOC ; // Cube
11      | load C tile mSlice  $\times$  nSlice to UB from LOC ; // Vector
12      | load C tile mSlice  $\times$  nSlice to External from UB ; // MTE3

```

and activates the second control flow. Fig. 13 illustrates how the Double Buffer policy fills the idleness gap of the MTE1 Unit to increase the concurrency and ILP. However, the Double Buffer policy sometimes introduces too much cost and lowers the performance because of the imbalanced execution period.

To formulate an optimization problem, we conduct the tiling parameter selection procedure as a permutation of all four parameters, which are the variables of the problem. The improved matrix multiplication has no further requirements or constraints for the variables, but the tiling numbers must be less than the $16 \times 16 \times 16$ block numbers along the same direction. For each parameter combination in the feasible domain, Verrocchio predicts the execution time and records, which performs the objective function of the optimization problem. To solve the problem, we implement the exhaustive search, the most straightforward solution. Finally, we

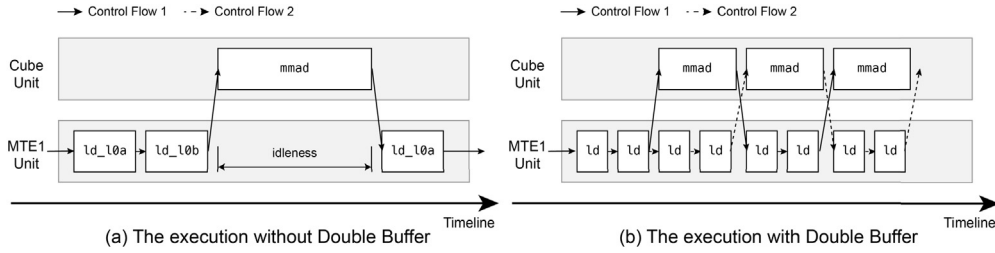


Fig. 13. The effect of Double Buffer policy on the execution performance and concurrency.

find the optimized solution as the parameter combination with the least execution time for our matrix multiplication.

6.2.2. Verrocchio accuracy results

To evaluate the Verrocchio accuracy, we compare our predicted results from Verrocchio with the practical execution time results, which adopt the optimized tiling parameters, as shown in Fig. 14. For a better visibility, we define the matrix multiplication scale as the number of the basic blocks operated by the Cube Unit ($m \times k \times n/16^3$). As some matrix multiplications can have the same matrix multiplication scale, we use the average values as the results.

Fig. 14 (a) shows the total execution time error rates. The average error rate among all test cases is 5.06% for the single-core execution and 5.25% for the double-core execution. The predicted results are lower than the experiment results when the scale is small. One of the potential reasons is that we do not consider the Scalar Unit and the corresponding instructions in Verrocchio. As we discussed in Sec. 4.2.1, the Scalar instructions could block the execution of other units. When the scale gets larger, the effect of the Scalar instructions declines apparently, and the error rates rise above the baseline. When the scale is larger than 5000, the error rates get stable at around 5%, meaning the prediction of Verrocchio is more accurate and stable in large-scale cases.

We also show their prediction error rate for each hardware unit separately in the following figures of Fig. 14. The Vector Unit and the Cube Unit are well-predicted, reporting the average error rates of 0.69% and 1.17% for the single-core execution, 0.51% and 0.85% for the double-core execution, shown in Fig. 14 (a) and (b) respectively. The MTE1 Unit performs slightly worse with the average error rates of 9.78% and 9.46%. We observe apparent error rates when the scale is small. One of the potential reasons is that the initialization time of the MTE1 instruction is less than other instructions. The MTE2 and MTE3 Units report the worst average error rates of 18.89% and 23.72% for the single-core execution, 14.22% and 33.72% for the double-core execution. Furthermore, the two MTE Units report a system error when the scale is large, where the predicted execution time is always larger than the practical experiment time, especially the MTE3 Unit results. One of the reasons is that, as shown in Fig. 7, the bus contentions of the MTE Units do not start immediately but wait for the finish of a block. Verrocchio considers all contentions starting at once when the instructions are called, which brings more contentions than the practical execution. In addition, we notice the two MTE Units perform worse in the double-core execution. According to the data collected in Table 2, the two DaVinci Cores of the Ascend 310 processors are not synchronized perfectly. Therefore, Verrocchio could bring false contentions in the double-core execution when two respective MTE Units on two cores do not fully overlap.

6.2.3. Matrix multiplications acceleration results

Operator level We compare the improved matrix multiplications with the Huawei CANN operators. As the CANN operators require different logical core numbers, we keep the active core numbers as their respective demands. Our improved matrix multiplication

always uses two cores, so we set the core number to two during the experiments of our algorithm. We test matrix multiplications in 12 shapes, from $W \times W \times 2W$ to $W \times W \times 6W$, as shown in Fig. 15.

The results show how Verrocchio helps the matrix multiplications to achieve a significant speedup. The improved matrix multiplications achieve an average speedup of 1.70 $\times$ among all test cases compared with the Huawei official CANN operators. Generally, the accelerations grow steadily with the increment of the matrix multiplication scale (or value W). For the regular-shaped matrix multiplications like $W \times W \times 2W$, our matrix multiplications show similar performance compared with the CANN operators, where the two approaches follow similar tiling strategies. However, our matrix multiplications perform much better for those irregular-shaped matrix multiplications like $W \times W \times 4W$. The main reason is that the CANN operators abuse the multiple logical cores with inefficient tilings in these irregular-shaped cases, even when the Ascend 310 processors have only two physical cores. Meanwhile, our improved matrix multiplications focus on the two physical DaVinci Cores with the most appropriate and efficient tilings suggested by Verrocchio, which make great improvements.

Furthermore, we plot our matrix multiplications in the roofline model [31], as shown in Fig. 16, which considers the Cube Unit FLOPS and the bandwidth between the L0 and L1 Buffer. Compared with the CANN operators, our matrix multiplication algorithm reports a smaller performance gap to the attainable roofline results. The average ratio of the peak hardware performance we achieve is 23.64%, 2.09X higher than that of the CANN operators. With the increment of the operation intensity, the ratio of our algorithm becomes higher than that of the CANN operators, and finally stops at about 38.78%, which further proves the better performance in larger scales we observed in Fig. 15.

Application level Currently, executing a DNN model on Ascend 310 processor needs an official ATC offered by Huawei, which converts the Caffe or TensorFlow model to the Huawei custom model. However, Huawei does not offer the structures and formats of the custom model. Hence, we propose an estimated method to evaluate the acceleration brought by the improved matrix multiplications for popular DNN applications. For each layer of a DNN application, we evaluate the total execution time and the Cube Unit execution time. We compute the matrix multiplication scales defined in the last subsection with the collected Cube Unit execution time. Therefore, with the average acceleration rates of the scale, we have an approximate acceleration result of this matrix multiplication operator. By estimating all operators, we have final results of the DNN application [12,1,28,22,8], as shown in Fig. 17.

Among all evaluation results, AlexNet shows the most significant speedup of 2.18 $\times$. The reason is that AlexNet is one of the earliest Convolutional Neural Networks (CNNs) with large Fully Connected (FC) layers. It requires matrix multiplications of a large scale. As we show in the last subsection, the CANN operators perform worse than our operators on a large scale. Generally, the DNN applications report an average speedup of 1.53 $\times$. Although YOLO

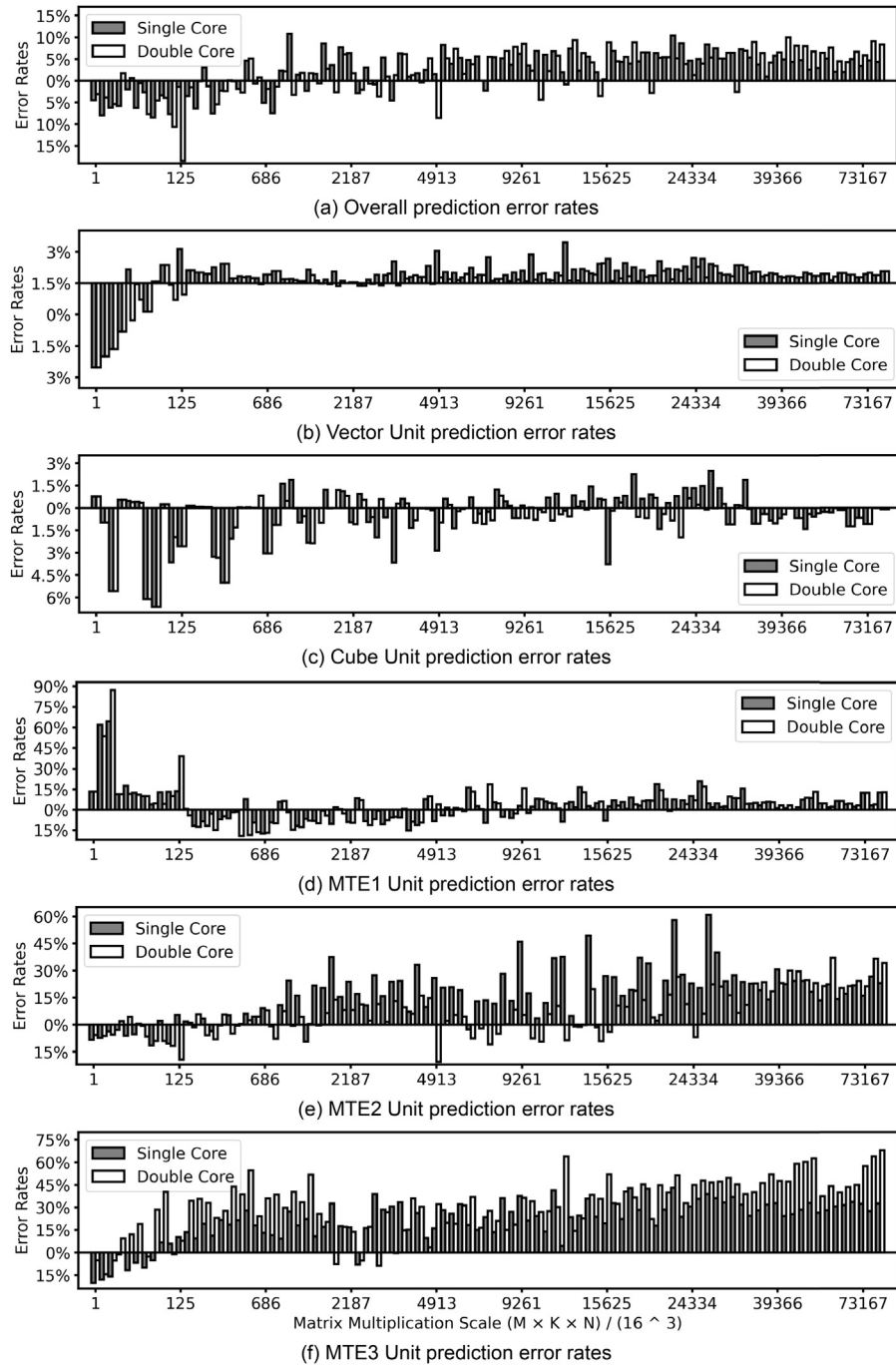


Fig. 14. The Verrocchio error rate results for the improved matrix multiplications.

V4 is a complex model, the matrix multiplication scales are not large, reporting the lowest acceleration rate of $1.15\times$.

7. Related work

With the growth of the AI processors, researchers paid high interest in this novel hardware with a lot of work in different areas. At the level of the applications, some studies focus on optimizing the original tasks of the AI processors, including the matrix multiplications [32] or the convolutions [33]. Other studies aim to extend the AI processors' computation power to more applications instead of deep learning only, e.g., the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm [18] or the reduction operation [6,27].

Meanwhile, some other works focus on exploring the hardware performance of the AI processors for general optimization. Similar to the previous works on regular GPUs [34], Jia et al. develop micro-benchmarking to dissect the architecture of the Tensor Cores [19]. Some other studies work on performance modeling on the AI processors, which can be mainly divided into two categories. One is the traditional way to do manual hardware featuring work and then model, which performs well on previous GPU work [35,14]. Raihan et al. [26] propose a Tensor Core microarchitecture model. They implement the model on a GPU simulator, GPGPU-sim, achieving 99.6% correlation versus the average instructions per cycle on an NVIDIA Titan V GPU for GEMM workloads. The other relies on the DNN models or other statistics results (e.g., GPU instruction sampling results in

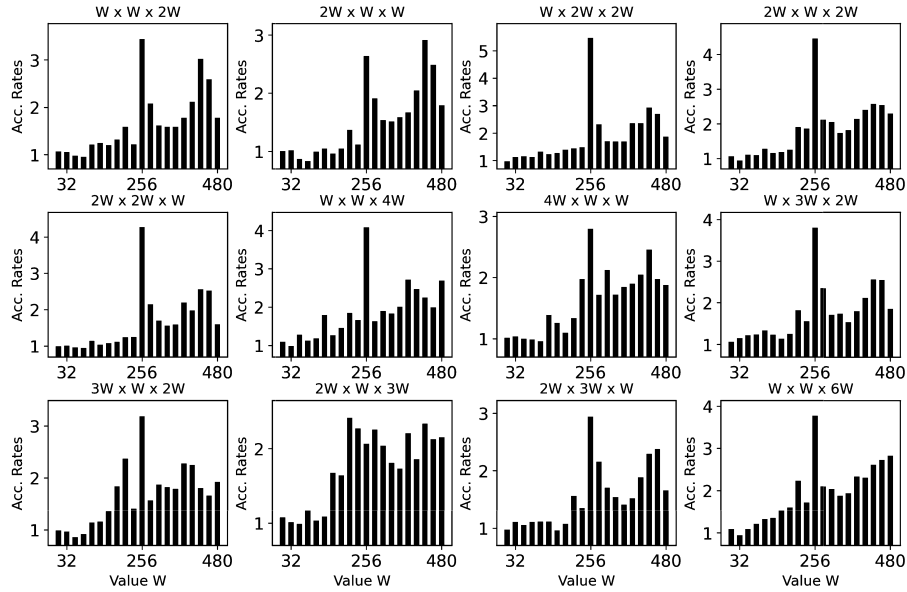


Fig. 15. Accelerations of the improved matrix multiplication compared with the CANN operators.

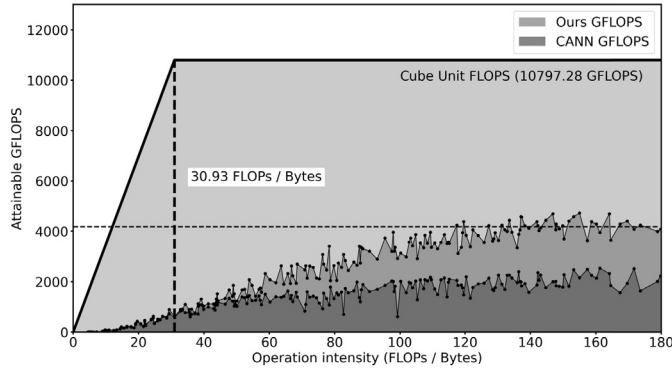


Fig. 16. The roofline model with the matrix multiplication results annotated.

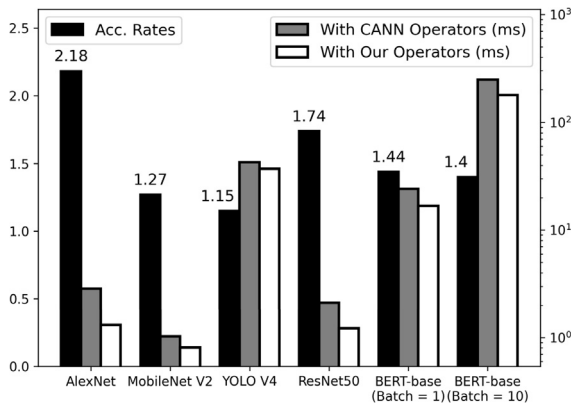


Fig. 17. The estimation results of the popular DNN applications.

previous work [36]) to predict the performance. Their models are more general but less explainable, which require no hardware details but large scales of previous execution records. Chen et al. [4] build a statistical cost model to estimate the cost of each low-level program for general tensor programs. They choose a multiple objectives function and train with transfer training to solve their formulated problem. Kaufman et al. [21] develop a learned performance model based on a Graph Neural Network (GNN) for tensor programs with 96.3% and 95.5% accuracy concerning true mea-

surements on Google TPUs. Our work follows the traditional way, focuses on the DaVinci Processor hardware details, and proposes an analytical white-box performance model.

8. Conclusion and future work

For the first time, we focus on building a performance model, Verrocchio, for Huawei DaVinci AI Cores. The DaVinci Core has complex data paths and distinct working details, which significantly influences the performance and brings a series of modeling challenges. Our discrete-event-based model reports high accuracy for various kernel programs but involves some system errors of the MTE Unit bandwidths, which are caused by the simplification of their runtime behaviors. In future work, we plan a deeper study with more benchmarks of the MTE Units, which aims to model the accurate bandwidth contentions.

CRediT authorship contribution statement

Yifeng Tang: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Cho-li Wang:** Conceptualization, Funding acquisition, Project administration, Resources, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

We would like to thank the editor and reviewers for their kind helps and significant suggestions for the publishing of this paper. This research was supported by Hong Kong RGC Research Impact Fund R5060-19.

References

- [1] A. Bochkovskiy, C. Wang, H.M. Liao, Yolov4: optimal speed and accuracy of object detection, CoRR, arXiv:2004.10934 [abs], 2020.
- [2] cambricon, Mu290-m5, <https://www.cambricon.com/index.php?m=content&c=index&a=list&catid=340/>. (Accessed 10 December 2021) (in Chinese).
- [3] Y. Chen, T. Chen, Z. Xu, N. Sun, O. Temam, Diannao family: energy-efficient hardware accelerators for machine learning, Commun. ACM 59 (11) (2016) 105–112, <https://doi.org/10.1145/2996864>.
- [4] T. Chen, L. Zheng, E.Q. Yan, Z. Jiang, T. Moreau, L. Ceze, et al., Learning to optimize tensor programs, in: Proceedings of the ACM International Conference on Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3–8, 2018, Montréal, Canada, 2018, pp. 3393–3404, <https://proceedings.neurips.cc/paper/2018/hash/8b5700012be65c9da25f49408d959ca0-Abstract.html>.
- [5] J. Choquette, W. Gandhi, O. Giroux, N. Stam, R. Krashinsky, NVIDIA A100 tensor core GPU: performance and innovation, IEEE MICRO 41 (2) (2021) 29–35, <https://doi.org/10.1109/MM.2021.3061394>.
- [6] A. Dakkak, C. Li, J. Xiong, I. Gelado, W.W. Hwu, Accelerating reduction and scan using tensor core units, in: Proceedings of the ACM International Conference on Supercomputing, ICS 2019, Phoenix, AZ, USA, June 26–28, 2019, ACM, 2019.
- [7] DelphiTools, Sampling profiler, <https://www.delphitools.info/samplingprofiler/>. (Accessed 14 July 2022).
- [8] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2–7, 2019, Volume 1 (Long and Short Papers), Association for Computational Linguistics, 2019, pp. 4171–4186.
- [9] E.W. Dijkstra, Over seinpalen, <http://www.cs.utexas.edu/users/EWD/ewd00xx/EWD74.PDF> (n.d.).
- [10] A. Gensler, J. Henze, B. Sick, N. Raabe, Deep learning for solar power forecasting—an approach using autoencoder and lstm neural networks, in: 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), IEEE, 2016, pp. 002858–002865.
- [11] Google, Profiling concepts, <https://cloud.google.com/profiler/docs/concepts-profiling>. (Accessed 14 July 2022).
- [12] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27–30, 2016, IEEE Computer Society, 2016, pp. 770–778.
- [13] J.L. Hennessy, D.A. Patterson, A new golden age for computer architecture, Commun. ACM 62 (2) (2019) 48–60, <https://doi.org/10.1145/3282307>.
- [14] C. Hong, A. Sukumaran-Rajam, J. Kim, P.S. Rawat, S. Krishnamoorthy, L. Pouchet, F. Rastello, P. Sadayappan, GPU code optimization using abstract kernel emulation and sensitivity analysis, in: J.S. Foster, D. Grossman (Eds.), Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018, Philadelphia, PA, USA, June 18–22, 2018, ACM, 2018, pp. 736–751.
- [15] C. Hristea, D. Lenoski, J.S. Keen, Measuring memory hierarchy performance of cache-coherent multiprocessors using micro benchmarks, in: Proceedings of the ACM/IEEE Conference on Supercomputing, SC 1997, November 15–21, 1997, San Jose, CA, USA, ACM, 1997, p. 45.
- [16] Huawei, Cann - a unified heterogeneous compute architecture, <https://www.huaweicloud.com/software/cann>. (Accessed 13 September 2021).
- [17] IBM, Sample-based profiling, <https://www.ibm.com/docs/en/mon-diag-tools?topic=perspective-sample-based-profiling>. (Accessed 14 July 2022).
- [18] Z. Ji, C. Wang, Accelerating DBSCAN algorithm with AI chips for large datasets, in: X. Sun, S. Shende, L.V. Kalé, Y. Chen (Eds.), ICPP 2021: 50th International Conference on Parallel Processing, Lemont, IL, USA, August 9–12, 2021, ACM, 2021, pp. 51:1–51:11.
- [19] Z. Jia, M. Maggioni, B. Staiger, D.P. Scarpazza, Dissecting the NVIDIA volta GPU architecture via microbenchmarking, CoRR abs/1804.06826, arXiv:1804.06826.
- [20] N.P. Jouppi, C. Young, N. Patil, D.A. Patterson, G. Agrawal, R. Bajwa, et al., In-datacenter performance analysis of a tensor processing unit, in: Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24–28, 2017, ACM, 2017, pp. 1–12.
- [21] S.J. Kaufman, P.M. Phothilimthana, Y. Zhou, M. Burrows, A learned performance model for the tensor processing unit, CoRR, arXiv:2008.01040 [abs], 2020.
- [22] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, in: Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3–6, 2012, Lake Tahoe, Nevada, United States, 2012, pp. 1106–1114, <https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>.
- [23] H. Liao, J. Tu, J. Xia, X. Zhou, Davinci: a scalable architecture for neural network computing, in: 2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18–20, 2019, IEEE, 2019, pp. 1–44.
- [24] J. Ouyang, X. Du, Y. Ma, J. Liu Kunlun, A 14 nm high-performance AI processor for diversified workloads, in: IEEE International Solid-State Circuits Conference, ISSCC 2021, San Francisco, CA, USA, February 13–22, 2021, IEEE, 2021, pp. 50–51.
- [25] Qualcomm, Snapdragon 888 ai presentation, <https://www.qualcomm.com/media/documents/files/snapdragon-888-ai-presentation.pdf>. (Accessed 2 October 2021).
- [26] M.A. Raihan, N. Goli, T.M. Aamodt, Modeling deep learning accelerator enabled gpus, in: IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2019, Madison, WI, USA, March 24–26, 2019, IEEE, 2019, pp. 79–92.
- [27] C.S. Rohwedder, J.P.L. de Carvalho, J.N. Amaral, G. Araújo, G. Colmanes, K.A. Wang, Pooling acceleration in the davinci architecture using im2col and col2im instructions, in: IEEE International Parallel and Distributed Processing Symposium Workshops, IPDPS Workshops 2021, Portland, OR, USA, June 17–21, 2021, IEEE, 2021, pp. 46–55.
- [28] M. Sandler, A.G. Howard, M. Zhu, A. Zhmoginov, L. Chen, Mobilenetv2: inverted residuals and linear bottlenecks, in: 2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18–22, 2018, Computer Vision Foundation / IEEE Computer Society, 2018, pp. 4510–4520, http://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html.
- [29] S. Srikanthan, S. Dwarkadas, K. Shen, Data sharing or resource contention: toward performance transparency on multicore systems, in: S. Lu, E. Riedel (Eds.), 2015 USENIX Annual Technical Conference, USENIX ATC '15, July 8–10, Santa Clara, CA, USA, USENIX Association, 2015, pp. 529–540, <https://www.usenix.org/conference/atc15/technical-session/presentation/srikanthan>.
- [30] Y. Tian, K. Pei, S. Jana, B. Ray, Deeptest: automated testing of deep-neural-network-driven autonomous cars, in: Proceedings of the 40th International Conference on Software Engineering, 2018, pp. 303–314.
- [31] S. Williams, A. Waterman, D.A. Patterson, Roofline: an insightful visual performance model for multicore architectures, Commun. ACM 52 (4) (2009) 65–76, <https://doi.org/10.1145/1498765.1498785>.
- [32] D. Yan, W. Wang, X. Chu, Demystifying tensor cores to optimize half-precision matrix multiply, in: 2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS), New Orleans, LA, USA, May 18–22, 2020, IEEE, 2020, pp. 634–643.
- [33] D. Yan, W. Wang, X. Chu, Optimizing batched winograd convolution on gpus, in: R. Gupta, X. Shen (Eds.), PPoPP '20: 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, San Diego, California, USA, February 22–26, 2020, ACM, 2020, pp. 32–44.
- [34] X. Zhang, G. Tan, S. Xue, J. Li, K. Zhou, M. Chen, Understanding the GPU microarchitecture to achieve bare-metal performance tuning, in: V. Sarkar, L. Rauchwerger (Eds.), Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, Austin, TX, USA, February 4–8, 2017, ACM, 2017, pp. 31–43.
- [35] K. Zhou, G. Tan, X. Zhang, C. Wang, N. Sun, A performance analysis framework for exploiting GPU microarchitectural capability, in: W.D. Gropp, P. Beckman, Z. Li, F.J. Cazorla (Eds.), Proceedings of the International Conference on Supercomputing, ICS 2017, Chicago, IL, USA, June 14–16, 2017, ACM, 2017, pp. 15:1–15:10.
- [36] K. Zhou, X. Meng, R. Sai, J.M. Mellor-Crummey, GPA: a GPU performance advisor based on instruction sampling, in: J.W. Lee, M.L. Soffa, A. Zaks (Eds.), IEEE/ACM International Symposium on Code Generation and Optimization, CGO 2021, Seoul, South Korea, February 27–March 3, 2021, IEEE, 2021, pp. 115–125.
- [37] M. Zitnik, F. Nguyen, B. Wang, J. Leskovec, A. Goldenberg, M.M. Hoffman, Machine learning for integrating data in biology and medicine: principles, practice, and opportunities, Inf. Fusion 50 (2019) 71–91.



Yifeng Tang received his BSc degree in computer science from Huazhong University of Science and Technology. He is currently a Ph.D. candidate in the department of Computer Science at the University of Hong Kong. His research interests include AI processor modeling, applications on AI processors, and AI processor optimization.



Cho-Li Wang received his Ph.D. degree in Computer Engineering from the University of Southern California in 1995. He is currently a professor at the Department of Computer Science at the University of Hong Kong. His research interests include parallel architecture (multicore, GPU, AI chips), software systems for Cloud Computing, and performance optimization of Distributed Deep Learning systems.